

RK0 User Manual

Using the embedded real-time kernel without surprises

Antonio Giacomelli

Draft for RK0 0.50.0 - 6 August 2026

Contents

Section	Topic
1	About this manual
2	Run RK0 on QEMU
3	A minimal RK0 application
4	Tasks and execution progress
5	Waiting, timeouts and execution context
6	Time services
7	Signals and waiting
8	Shared-state coordination
9	Buffered message passing
10	Synchronous message passing
11	Latest-state publication
12	Deterministic memory
13	Dynamic tasks
14	Trace and fault handling
15	Choosing a communication service
16	Compact API index
17	Further reading

1 About this manual

This manual explains how to use RK0 services and what an application observes when it uses them. It is deliberately shorter and more operational than the RK0 DocBook.

For each service, the manual answers three questions:

1. What relationship does the service express?
2. What does a successful call mean to the caller?
3. When should the service be selected instead of a nearby alternative?

The public prototypes and exhaustive return values remain authoritative in `core/inc/kapi.h`. The [RK0 DocBook](#) explains design rationale and implementation. The [Service Map](#) compares the contracts.

This draft describes RK0 0.50.0 at repository commit [2c801c4](#). In this version:

- a Mailbox is a one-slot Message Queue;
- a Port is a Message Queue bound to one owner task;
- a Channel endpoint belongs to its server task; there is no `RK_CHANNEL` object;
- a Rendezvous endpoint belongs to its receiver task; there is no `RK_RENDEZVOUS` object.

1.1 Notation

RK0 priorities run in the direction used by many fixed-priority kernels:

Priority 0 is the highest user priority. A smaller number means a more urgent task.

The figures use the following vocabulary:

- **RUNNING** - the task currently owns the processor;
- **READY** - the task may be selected by the scheduler;
- **WAITING** - the task cannot run until a time or coordination condition is satisfied;
- a solid arrow - an operation or state transition;
- a dashed arrow - a notification or return path that does not carry stored history.

1.2 Return values

RK0 separates successful operations, well-defined unsuccessful outcomes and invalid use:

Value	Meaning	Examples
0	Operation completed	<code>RK_ERR_SUCCESS</code>
positive	Valid operation, condition not met	queue full, timeout, no waiter
negative	Invalid object, parameter, state or context	null object, wrong owner, ISR misuse

Do not treat every non-zero result as a kernel fault. A non-blocking receive from an empty queue is a normal observation. In contrast, unlocking another task's mutex is invalid use.

Argument and context checks depend on `RK_CONF_ERR_CHECK`. Application code must honour the API preconditions even when a release build does not diagnose every invalid call.

2 Run RK0 on QEMU

RK0 currently builds with the GNU Arm Embedded toolchain and runs the repository demonstration on QEMU.

Required commands:

```
git clone https://github.com/antoniogiacomelli/RK0.git
cd RK0
make ARCH=armv7m qemu
```

Use `ARCH=armv6m` for the Cortex-M0 QEMU target. The default application is `app/src/application.c`. It contains selectable examples intended as executable documentation.

The principal build outputs are placed below `build/<architecture>/`:

- `rk0_demo.elf` - executable and debugger image;
- `rk0_demo.bin` - raw binary;
- `rk0_demo.hex` - Intel HEX image;
- `rk0_demo.map` - linker map.

For a debug server instead of immediate execution:

```
make ARCH=armv7m qemu-debug
```

QEMU then waits for GDB on TCP port 1234.

3 A minimal RK0 application

An RK0 application provides ordinary platform initialisation, static kernel storage and a mandatory `kApplicationInit()` function.

```
#include <kapi.h>

#define WORKER_STACK_WORDS 160U
#define WORKER_PRIORITY    2U

RK_DECLARE_TASK(workerHandle, WorkerTask,
                workerStack, WORKER_STACK_WORDS)

int main(void)
{
    kCoreInit();
    kInit();

    /* kInit() does not return during normal operation. */
    while (1)
    {
        kErrorHandler(RK_FAULT_APP_CRASH);
    }
}

VOID kApplicationInit(VOID)
{
    RK_ERR err = kCreateTask(&workerHandle,
                           WorkerTask,
                           RK_NO_ARGS,
                           "Worker",
                           workerStack,
                           WORKER_STACK_WORDS,
                           WORKER_PRIORITY,
                           RK_PREEMPT);
    K_ASSERT(err == RK_ERR_SUCCESS);
}

VOID WorkerTask(VOID *args)
{
    RK_UNUSEARGS

    while (1)
    {
        DoOneCycle();
        kSleepPeriodic(RK_MS_TO_TICKS(20U));
    }
}
```

`RK_DECLARE_TASK` declares the entry prototype, an 8-byte-aligned stack and a task handle. Stack size is expressed in 32-bit words, not bytes.

`kInit()` calls `kApplicationInit()`, completes kernel initialisation, makes the created tasks ready, enables interrupts and starts the highest-priority task. Create startup tasks and initialise the objects they need inside `kApplicationInit()`.

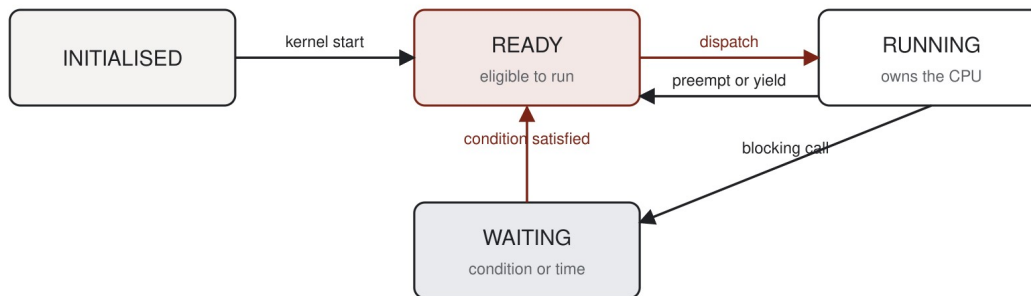
Keep task storage alive. A static task's handle and stack must remain valid for the life of the application. File-scope storage is the normal choice.

4 Tasks and execution progress

4.1 Task states

The scheduler is the only component that changes a task from READY to RUNNING.

Only the scheduler changes READY to RUNNING



RKO task lifecycle

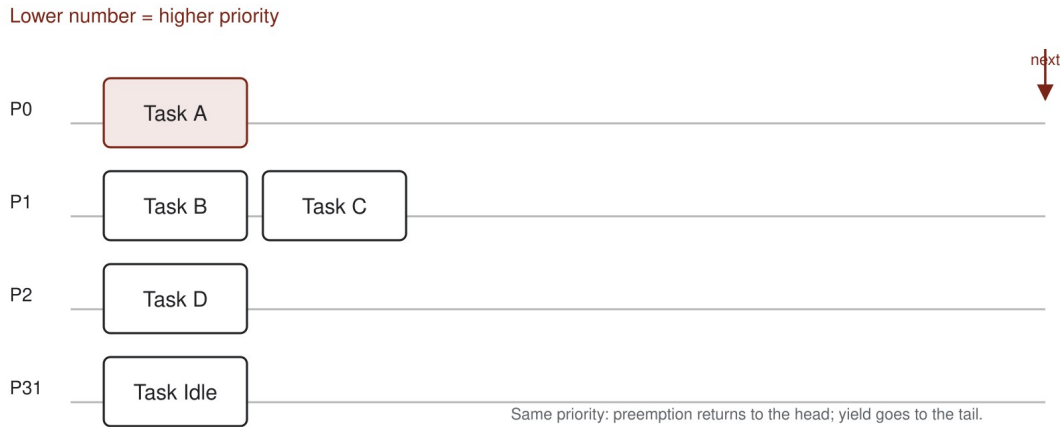
A static task normally moves among INITIALIZED, READY, RUNNING and a family of WAITING states. A dynamic task may additionally become TERMINATED.

- A task becomes **READY** when creation completes after scheduler start, or when its waiting condition is satisfied.
- The scheduler changes one READY task to **RUNNING**.
- A higher-priority ready task may preempt a preemptible running task.
- `kYield()` voluntarily returns the caller to READY.
- A blocking service changes the caller to a service-specific WAITING state.

Returning from a task entry function is not a task-termination operation. Task functions should normally contain a loop. Dynamic tasks terminate through the dynamic-task API described later.

4.2 Fixed-priority FIFO scheduling

RKO maintains one FIFO ready queue per priority and does not perform automatic time slicing.



Fixed-priority FIFO ready queues

The highest-priority non-empty ready queue wins. Within the same priority:

- initial order follows task creation order;
- a preempted task returns to the head, so it resumes before equal-priority peers;
- a yielded task goes to the tail;
- a task made ready after waiting goes to the tail.

Two equal-priority tasks therefore cooperate by yielding or waiting. If the running task does neither, there is no expressed reason for another equal-priority task to run.

`RK_NO_PREEMPT` creates an exceptional non-preemptible task. Once dispatched, it continues until it blocks, yields or otherwise leaves RUNNING. Use this only for short, bounded service routines: it can delay every user task, including more urgent ones.

4.3 Yielding is not sleeping

`kYield()` says that the task has completed its present turn among ready peers. It does not express a time condition and does not make the task WAITING.

If no equal- or higher-priority task is ready, yielding does not manufacture work for a lower-priority task. The caller remains the correct task to run.

4.4 Scheduler lock

`kSchLock()` and `kSchUnlock()` (also exposed as `kPreemptDisable()` and `kPreemptEnable()`) defer user-task preemption. Locks are nested and belong to the task.

The scheduler lock does **not** disable interrupts and is not a mutex. It does not establish shared-resource ownership or priority inheritance. Use it only where delaying dispatch is itself the required, bounded operation.

5 Waiting, timeouts and execution context

Most services that may wait accept one of three timeout forms:

Timeout	Caller behaviour
<code>RK_NO_WAIT</code>	Inspect or attempt the operation and return immediately
finite ticks	Wait up to the requested kernel time
<code>RK_WAIT_FOREVER</code>	Wait until the service condition is satisfied

Finite values must not exceed `RK_MAX_PERIOD`. Convert deliberately between milliseconds and ticks:

```
RK_TICK ticks = RK_MS_TO_TICKS(25U);
```

The conversion truncates. A duration shorter than one kernel tick becomes zero and therefore changes the operation into `RK_NO_WAIT` for timeout-bearing APIs.

5.1 What a timeout guarantees

A timeout removes a task's dependency on the object and makes it READY. Actual return to application code still depends on fixed-priority scheduling. A newly ready task cannot execute while a more urgent task legitimately remains RUNNING.

The operation's return code tells the caller which completion occurred. After `RK_ERR_TIMEOUT`, application code must assume that the requested operation did not complete unless that service explicitly defines an abandoned in-progress state, as Channel does.

5.2 Interrupt service routines

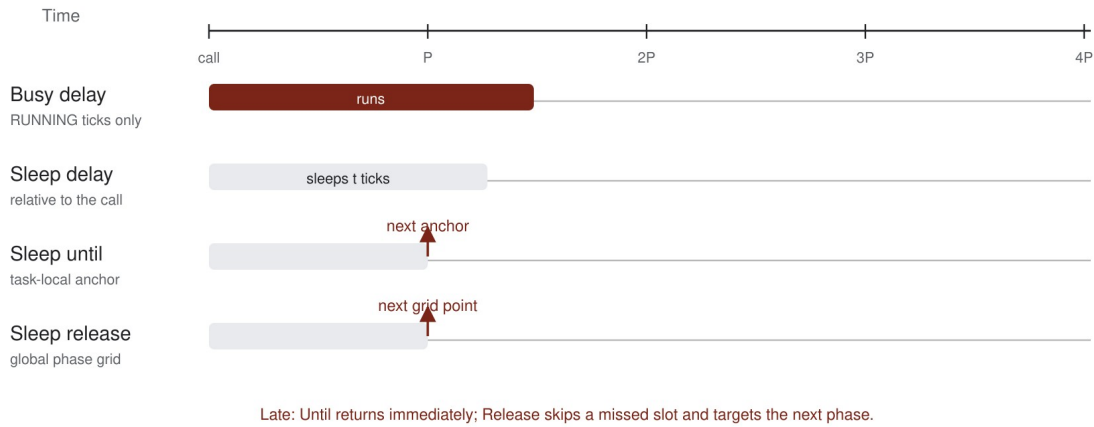
An ISR must never request an operation that can block. Some services provide a non-blocking ISR path, such as `kEventSet()`, `kSemaphorePost()` and no-wait Message Queue operations. Multi-task wake operations may be deferred to the PostProc system task.

Task-backed endpoints - Port receive, Channel and Rendezvous - should be used from task context. Their meaning depends on the current task, not only on whether the implementation can reject a particular call.

When ISR use matters, check the allowed context in `kapi.h` for the exact operation. Do not infer ISR safety merely from the absence of a timeout parameter.

6 Time services

RK0 keeps different meanings of time in separate calls.



Comparison of RK0 time services

6.1 Choose the intended time relation

Service	Meaning	Best use
<code>kBusyDelay(t)</code>	Consume t ticks while RUNNING	Simulated or deliberately active workload
<code>kSleepDelay(t)</code>	Sleep relative to this call	One-off delay or back-off
<code>kSleepUntil(&anchor, p)</code>	Advance one task-local deadline by p	Periodic work that must account for every activation
<code>kSleepRelease(p)</code>	Target the next scheduler-start phase slot	Phase-aligned periodic work that may skip missed releases

6.2 Busy Delay

`kBusyDelay(t)` leaves the caller RUNNING. Only ticks observed while the caller executes count towards completion. Time spent preempted does not reduce the remaining busy delay.

Use it to represent CPU work in an example or for a genuinely active wait. It does not free the processor for a lower-priority task.

6.3 Sleep Delay

`kSleepDelay(t)`, also named `kSleep(t)`, suspends the task for a relative duration measured from the call.

```
DoWork();
kSleep(RK_MS_T0_TICKS(50U));
```

In a loop, work time, preemption and release jitter accumulate. Do not use relative sleep to define a periodic task.

6.4 Sleep Until

`kSleepUntil()` uses a reference owned by the task:

```
RK_TICK anchor = kTickGet();
while (1)
{
    AcquireAndProcessSample();
    RK_ERR err = kSleepUntil(&anchor,
                           RK_MS_TO_TICKS(10U));
    if (err == RK_ERR_ELAPSED_PERIOD)
    {
        RecordLateActivation();
    }
}
```

Each call advances the anchor by one period. If the deadline has already elapsed, the call returns immediately with `RK_ERR_ELAPSED_PERIOD`. It does not skip forward over multiple releases. This favours execution count over absolute phase recovery.

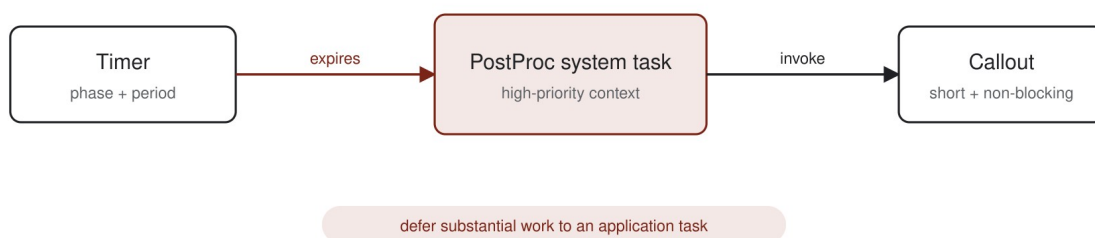
6.5 Sleep Release

`kSleepRelease()`, also named `kSleepPeriodic()`, uses the phase grid established when scheduling begins. A late task sleeps less to reach the next phase. If an entire release has been missed, RK0 records an overrun, skips the missed slot and targets the next valid one.

Use it for known startup tasks that should remain aligned to a system phase. Do not use the global startup phase of `kSleepRelease()` for a task spawned later at an arbitrary time; use a task-local `kSleepUntil()` anchor instead.

6.6 Application timer callouts

An application timer defers a callback from tick expiry to the PostProc system task.



Application timer callout context

`kTimerInit()` arms a one-shot or reloading timer with an initial phase, a period, a callback and an argument pointer. `kTimerCancel()` cancels an active timer.

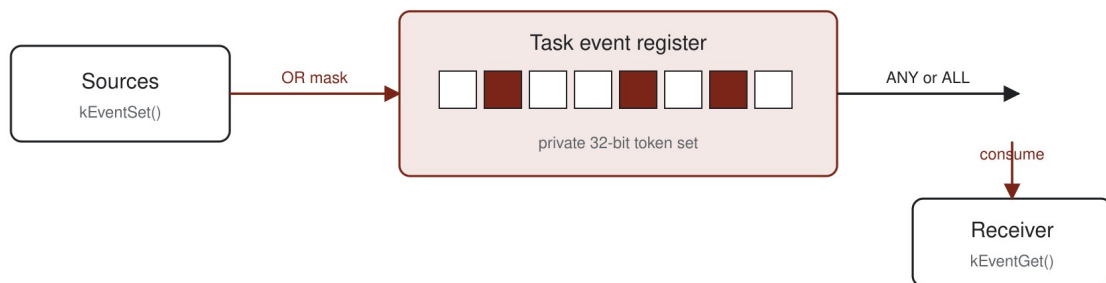
Callouts execute in high-priority, non-preemptible system-task context. They must be short and non-blocking. A normal pattern is to set a task event or post a semaphore and let an application task perform the work.

7 Signals and waiting

Signals express occurrence or availability. They do not carry an application payload.

7.1 Task Event Register

Every task has a private 32-bit Event Register. Other tasks or an ISR set bits; only the receiving task waits for and consumes them.



Repeated sets of the same bit collapse into one pending token.

Task Event Register

`kEventSet(task, mask)` ORs the supplied mask into the current register. Setting the same bit repeatedly before it is consumed still leaves one bit set. If each occurrence matters, use a counting Semaphore or a Message Queue.

The receiver selects `RK_EVENT_ANY` or `RK_EVENT_ALL`:

```
#define EVT_SAMPLE    RK_EVENT_1
#define EVT_SETPoint RK_EVENT_2

RK_EVENT_FLAG got = 0UL;

RK_ERR err = kEventGet(EVT_SAMPLE | EVT_SETPoint,
                      RK_EVENT_ALL,
                      &got,
                      RK_WAIT_FOREVER);
K_ASSERT(err == RK_ERR_SUCCESS);
```

On success, RK0 optionally copies the complete register state to `got`, then clears the requested mask. Bits outside that mask remain pending.

Use Task Events when:

- the receiver is known;
- no payload is needed;
- bitwise ANY or ALL composition is useful;
- repeated occurrences may coalesce.

`kEventQuery()` observes the current register. `kEventClear()` explicitly clears selected bits. Passing `NULL` as their task handle selects the calling task; an ISR must name a target task explicitly.

7.2 Semaphores

A Semaphore stores an unsigned count of available units or pending occurrences. It has no owner.

```
static RK_SEMAPHORE samplesReady;

VOID kApplicationInit(VOID)
{
    RK_ERR err = kSemaphoreInit(&samplesReady,
                                0U, /* initially no sample */
                                8U); /* retain up to 8 */
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

`kSemaphorePend()` consumes a stored count. If the count is zero, it returns immediately for `RK_NO_WAIT` or places the task in the priority-ordered wait queue.

`kSemaphorePost()` first satisfies the most urgent waiting task. With no waiter, it increments the count up to the configured maximum. Posting at the maximum leaves the value unchanged and returns `RK_ERR_SEMA_FULL`.

Convenience initialisers are available:

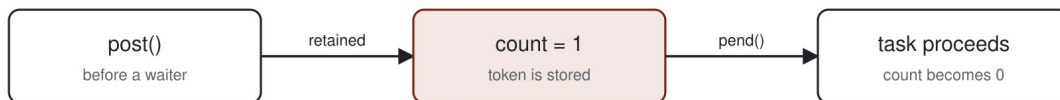
```
kSemaBinInit(&doorbell, 0U); /* maximum 1 */
kSemaCountInit(&credits, 12U); /* maximum UINT */
```

A binary Semaphore can guard a simple region only by application convention. It is not a Mutex: any task may post it, there is no owner and priority inversion is not handled.

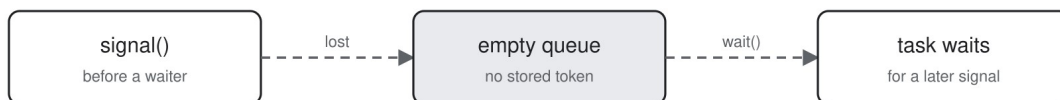
7.3 Sleep Queues

A Sleep Queue is a pure, non-latching waiting relation. It stores waiting tasks, not a condition and not an occurrence token.

Semaphore



Sleep Queue



Semaphore token retention compared with a Sleep Queue

- `kSleepQueueWait()` starts waiting from the call onward.
- `kSleepQueueSignal()` wakes the highest-priority waiter.
- `kSleepQueueWake(q, n, left)` wakes up to `n` waiters; `n == 0` means all.
- `kSleepQueueReady()` wakes a named task from that exact queue.
- `kSleepQueueBlockReadyTask()` moves a non-running READY task to the queue.

8 Shared-state coordination

8.1 Mutex Locks

A Mutex expresses exclusive ownership of shared state. Once locked, only its owner may unlock it. RK0 Mutexes are non-recursive: attempting to lock the same Mutex twice is invalid.

```
static RK_MUTEX stateLock;

VOID kApplicationInit(VOID)
{
    RK_ERR err = kMutexInit(&stateLock,
                           RK_PRIO_INHERITANCE);
    K_ASSERT(err == RK_ERR_SUCCESS);
}

VOID UpdateState(VOID)
{
    RK_ERR err = kMutexLock(&stateLock, RK_WAIT_FOREVER);
    K_ASSERT(err == RK_ERR_SUCCESS);

    sharedState.value++;

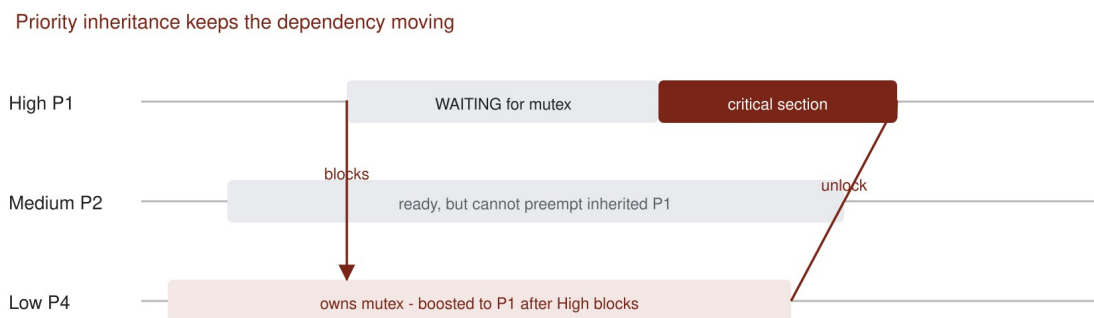
    err = kMutexUnlock(&stateLock);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

The protocol is selected at initialisation:

- **RK_PRIO_NONE** - mutual exclusion only;
- **RK_PRIO_INHERITANCE** - transitive priority inheritance.

8.1.1 Priority inheritance

When a high-priority task waits for a Mutex owned by a lower-priority task, the owner inherits the waiter's effective priority. This prevents unrelated medium-priority work from extending the inversion.



Transitive priority inheritance on a Mutex

RK0 recomputes a task's effective priority across all PI-enabled Mutexes it owns. If an owner itself waits on another Mutex, the inherited priority propagates through the dependency chain.

Priority inheritance bounds interference; it does not make long critical sections inexpensive. Keep the protected region bounded and do not perform unrelated blocking operations while owning a Mutex.

8.1.2 Mutex versus binary Semaphore

Property	Mutex	Binary Semaphore
Stored state	locked/unlocked	count 0/1
Owner	yes	no
Who may release	owner only	any task or ISR permitted by API
Priority inheritance	optional	no
Intended meaning	exclusive shared-state access	occurrence or availability

8.2 Condition-variable helpers

RK0 provides helpers that compose a Sleep Queue with a Mutex:

```
while (!PredicateIsTrue())
{
    RK_ERR err = kCondVarWait(&condition,
                             &stateLock,
                             RK_WAIT_FOREVER);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

The caller tests an application predicate while owning the Mutex. Waiting releases the Mutex and joins the Sleep Queue without allowing another task to slip between those two actions. After a successful wake, the helper reacquires the Mutex before returning.

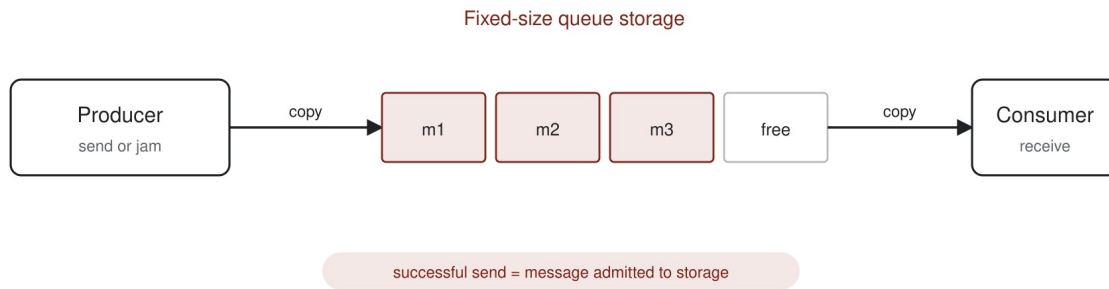
Always retest the predicate in a loop. A wake means that the condition may have changed; it is not proof that this particular task may proceed.

`kCondVarSignal()` wakes one waiter. `kCondVarBroadcast()` wakes all. The application decides which signalling discipline preserves its invariant.

9 Buffered message passing

9.1 Message Queues

A Message Queue transfers fixed-size messages by copy and preserves their order up to the configured queue capacity.



Capacity absorbs bursts. It does not increase long-term throughput.

Fixed-size Message Queue

Use a Message Queue when every accepted message remains relevant and a bounded buffer should absorb short differences between producer and consumer timing.

A successful send means one of two things:

- the message was copied directly into an already-waiting receiver's buffer; or
- the message was copied into queue storage.

It does not mean that the receiver has processed the message.

9.1.1 Declare and initialise

```

typedef struct
{
    ULONG timestamp;
    UINT value;
} Sample;

#define SAMPLE_DEPTH 4U

static RK_MSG_QUEUE sampleQueue;
RK_DECLARE_MSG_QUEUE_BUF(sampleQueueStorage,
                          Sample,
                          SAMPLE_DEPTH)

VOID InitSampleQueue(VOID)
{
    RK_ERR err = kMsgQueueInit(&sampleQueue,
                              sampleQueueStorage,
                              RK_MSGQ_MSG_SIZE(Sample),
                              SAMPLE_DEPTH);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
  
```

Message size is rounded to one of the supported sizes: 1, 2, 4 or 8 words. The declaration macro allocates matching word-aligned storage. Payloads larger than 8 words should normally be placed in a fixed-block Memory Partition and transferred through a one-word queue of pointers.

9.1.2 Send and receive

```
Sample sample = ReadSample();

RK_ERR err = kMsgQueueSend(&sampleQueue,
                          &sample,
                          RK_NO_WAIT);
if (err == RK_ERR_BUFFER_FULL)
{
    RecordDroppedSample();
}
```

`kMsgQueueSend()` appends. `kMsgQueueJam()` inserts at the front. A sender may wait when the queue is full. A receiver may wait when it is empty. Waiting senders and receivers are selected by effective priority, FIFO among equal priorities.

`kMsgQueuePeek()` copies the front message without consuming it. `kMsgQueueReset()` discards buffered messages and releases current waiters. `kMsgQueueQuery()` reports buffered messages and waiting senders or receivers.

A queue does not create throughput. Capacity absorbs bursts and jitter. Over time, either the producer waits, messages are intentionally dropped, or the consumer must sustain the arrival rate.

9.1.3 Send callback

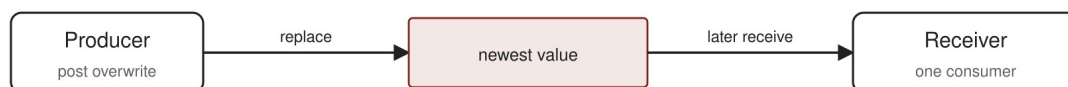
When enabled, `kMsgQueueInstallSendCb()` installs a callback after a successful normal send or jam. The callback runs synchronously inside the sending operation. It must be short and non-blocking, normally setting a Task Event or posting a Semaphore.

The callback is notification, not delivery. The queue remains the authority for the message.

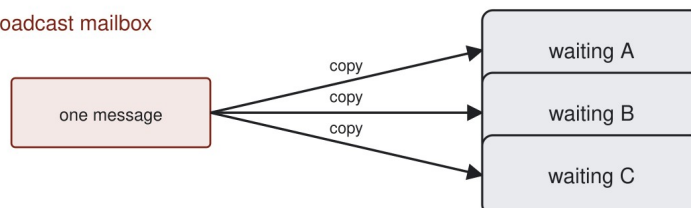
9.2 Mailboxes

`RK_MBOX` is an alias for a Message Queue whose depth is exactly one. Ordinary post and pend operations retain queue semantics.

Overwrite mailbox



Broadcast mailbox



Only tasks already blocked in broadcast receive are targeted.

One-slot Mailbox operations

9.2.1 Overwrite

`kMsgQueuePost0vw()` replaces the pending value instead of waiting for the slot to become free. Use this when only the newest not-yet-consumed value matters to one receiver.

Overwrite is not the same as MRM. A Mailbox has one slot and normal receivers compete for it; MRM retains versions safely while multiple readers finish independently.

9.2.2 Broadcast

Tasks first block in `kMboxBroadcastRecv()`. A later `kMboxBroadcast()` copies one message to every task that is already waiting in broadcast receive.

If no broadcast receiver is waiting, no message is deposited and the current API returns `RK_ERR_BUFFER_EMPTY`. A broadcast is therefore a simultaneous handoff to present waiters, not retained publication for future readers.

9.3 Ports

A Port is a Message Queue bound to one owner task. It is not another kernel object.

```
static RK_MSG_QUEUE dacPort;
RK_DECLARE_PORT_BUF(dacPortStorage, DacRequest, 3U)

VOID InitDacPort(VOID)
{
    RK_ERR err = kPortInit(&dacPort,
                          dacPortStorage,
                          RK_MSGQ_MSG_SIZE(DacRequest),
                          3U,
                          dacManagerHandle);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

Clients name the owner task when sending:

```
kPortSend(dacManagerHandle, &request, RK_WAIT_FOREVER);
```

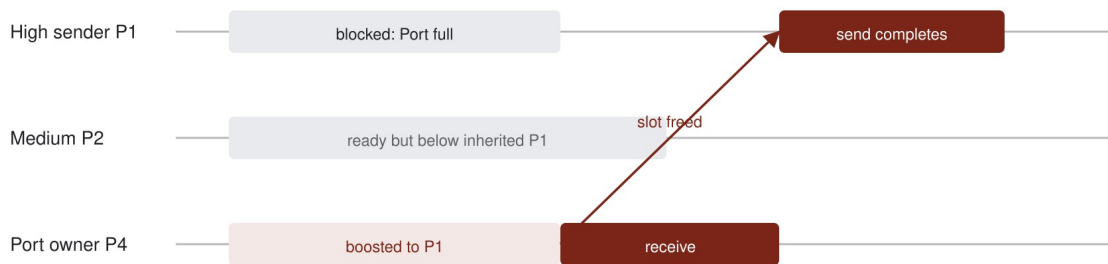
The owner receives from its own endpoint:

```
kPortRecv(&request, RK_WAIT_FOREVER);
```

Only the owner may receive. A task can own one Port.

9.3.1 Backpressure and owner priority

When the Port is full and a sender blocks, the owner can inherit the most urgent blocked sender priority so that it can receive and relieve the full-queue dependency.



The boost ends when backpressure is relieved - not when the work is processed.

Port backpressure and owner boost

No boost occurs merely because a message is buffered. Successful `kPortSend()` still means only that the message was admitted. If a client needs an answer or completion result, use a Channel or design an explicit return path.

9.3.2 Single coordination authority

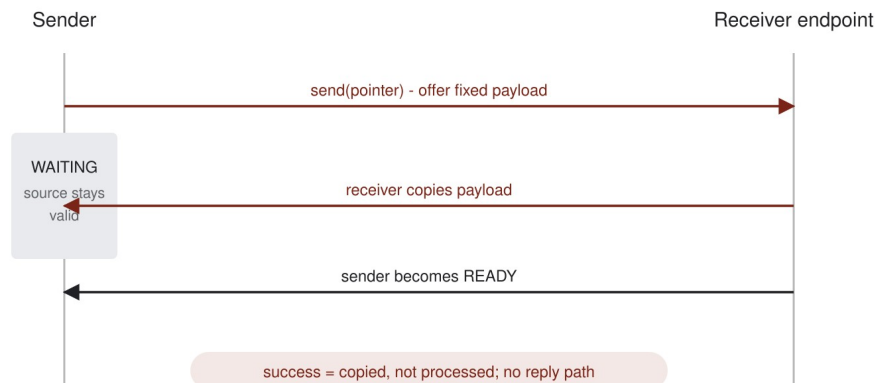
Do not expose the same resource through both direct Mutex-protected access and a Port-owned server path. Those are competing ownership authorities and can form dependency cycles that priority inheritance cannot safely resolve.

RK0 enforces the rule on ownership-transfer paths: Port send, receive, jam, overwrite and reset reject a running task that owns any Mutex.

10 Synchronous message passing

10.1 Rendezvous

Rendezvous is unbuffered task-to-task message passing. The sender names a receiver and waits until that receiver copies the fixed-size payload.



Unbuffered Rendezvous handoff

The receiver task owns an implicit endpoint configured once:

```

typedef struct
{
    UINT sequence;
    UINT sample;
} Sample;

RK_ERR err = kRendezvousInit(controllerHandle,
                             sizeof(Sample));
K_ASSERT(err == RK_ERR_SUCCESS);
  
```

The byte size must be non-zero and a multiple of `RK_WORD_SIZE`. There is no standalone Rendezvous object and no kernel payload buffer.

10.1.1 Send-and-wait-until-copied

```

Sample sample = MakeSample();

RK_ERR err = kRendezvousSend(controllerHandle,
                             &sample,
                             RK_WAIT_FOREVER);
  
```

If the receiver is waiting, the payload is copied immediately. Otherwise, the sender waits and its source pointer remains part of the pending offer. Multiple senders may wait, selected by effective priority, but their messages are not buffered in a queue.

The receiver calls:

```

Sample sample;

RK_ERR err = kRendezvousRecv(&sample,
                             RK_WAIT_FOREVER);
  
```

A successful send means that the receiver copied the payload into its own storage. It does not mean that the receiver processed it, and there is no reply path.

When an urgent sender waits for a less-urgent receiver, the receiver inherits the highest waiting sender priority until the handoff dependency is removed.

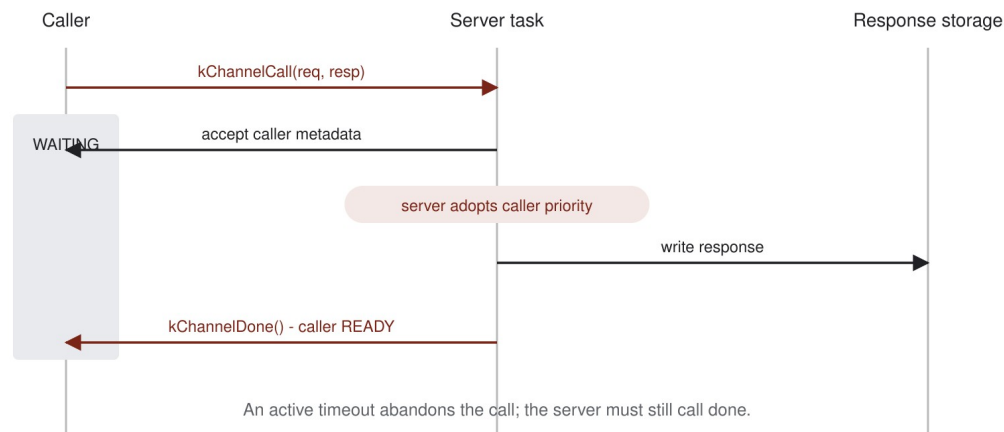
Buffer lifetime: the sender's source storage must remain valid until `kRendezvousSend()` returns. A sender timeout invalidates the offer, so a later receiver cannot consume a stale pointer.

Use Rendezvous when delivery acknowledgement is sufficient. Use a Channel when the caller needs a computed response. Use a Port or Message Queue when backlog is part of the contract.

Rendezvous send and receive reject a task that owns a Mutex, preserving the single-authority rule.

10.2 Call Channels

A Channel is a synchronous N:1 procedure call to a server task. The client provides request and response storage, calls the server and waits for `kChannelDone()`.



Channel request, accept and completion

There is no `RK_CHANNEL` object and no initialisation call. Every task has an implicit server endpoint. Pending call metadata lives in the caller's Task Control Block; accepted metadata is copied into stack-local `RK_CALL_DATA` on the server.

10.2.1 Client

```

ChannelRequest request = {
    .sample = 123U,
    .scale = 3U
};
ChannelResponse response = {0U};
RK_ERR err = kChannelCall(serverHandle,
    &request,
    &response,
    sizeof(request),
    RK_WAIT_FOREVER);
if (err == RK_ERR_SUCCESS)
{
    UseResult(response.result);
}
  
```

`kChannelCall()` always waits; `RK_NO_WAIT` is not a valid Channel call timeout.

10.2.2 Server

```
while (1)
{
    RK_CALL_DATA call = {0};

    RK_ERR err = kChannelAccept(&call,
                               RK_WAIT_FOREVER);
    K_ASSERT(err == RK_ERR_SUCCESS);

    ChannelRequest const *request = call.reqPtr;
    ChannelResponse *response = call.respPtr;

    response->result = request->sample * request->scale;

    err = kChannelDone(&call);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

Many callers may wait on one server, ordered by effective priority. Only one call may be active. After accepting a call, the server adopts the caller's effective priority until `kChannelDone()`.

Successful client completion means that the server called `kChannelDone()`. The kernel does not interpret the request or response payload.

10.2.3 Timeout after acceptance

A queued timeout removes the caller before acceptance. A timeout after acceptance creates an **abandoned call**:

- the caller returns `RK_ERR_TIMEOUT`;
- the server may still be executing;
- the server must still call `kChannelDone()`;
- the caller cannot start another Channel call until that completion clears the abandoned call.

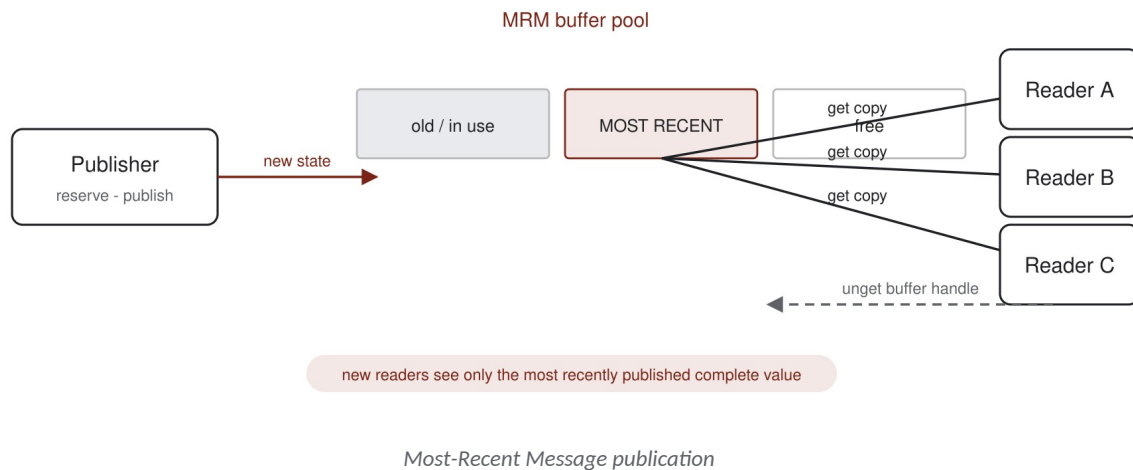
The accepted `RK_CALL_DATA` contains raw application pointers. If a bounded call can time out after acceptance, request and response storage must remain valid until the server calls `kChannelDone()`. Caller-stack objects are therefore unsafe for that case unless their lifetime is otherwise extended.

`kChannelCall()`, `kChannelAccept()` and `kChannelDone()` reject a running task that owns a Mutex. Always pair every successful accept with done, including abandoned calls.

11 Latest-state publication

11.1 Most-Recent Message

MRM is a non-blocking 1:N protocol for publishing the latest complete state. New readers never walk through a backlog of obsolete samples.



Use it when:

- one writer publishes state;
- several readers operate at independent rates;
- the latest complete value is more relevant than every intermediate value;
- readers must never observe a partially updated object.

11.1.1 Configure both pools

MRM uses two equal-length pools:

1. `RK_MRM_BUF` control buffers, which hold reader counts and payload addresses;
2. application payload buffers.

For one writer and R readers, configure $R + 2$ elements: one per communicating task plus one spare. For an application type `PlantState`:

```
#define N_READERS 3U
#define N_MRM_BUFS (N_READERS + 2U)

static RK_MRM stateMrm;
static RK_MRM_BUF stateControlPool[N_MRM_BUFS];
static PlantState statePayloadPool[N_MRM_BUFS] K_ALIGN(4);

RK_ERR err = kMRMInit(&stateMrm, stateControlPool,
                     statePayloadPool, N_MRM_BUFS,
                     RK_TYPE_WORD_COUNT(PlantState));
K_ASSERT(err == RK_ERR_SUCCESS);
```

Use static zero-initialised storage for the `RK_MRM_BUF` pool.

11.1.2 Writer: reserve and publish

```
RK_MRM_BUF *buffer = kMRMReserve(&stateMrm);
if (buffer != NULL)
{
    PlantState state = ReadPlantState();
    RK_ERR err = kMRMPublish(&stateMrm,
                           buffer,
                           &state);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

Reservation is private to the writer until publication. Publishing copies the payload and makes that buffer the only version visible to new readers.

11.1.3 Reader: get and unget

```
PlantState state;
RK_MRM_BUF *buffer = kMRMGet(&stateMrm, &state);

if (buffer != NULL)
{
    ProcessState(&state);

    RK_ERR err = kMRMUnget(&stateMrm, buffer);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

`kMRMGet()` copies the current payload and returns a handle to the version from which it was taken. Pair every successful get with exactly one unget of that same handle. The handle lets RK0 retain an old version while readers that acquired it are still active.

MRM never waits, wakes a task or donates priority. `kMRMReserve()` returns `NULL` when no buffer is available; `kMRMGet()` returns `NULL` before the first publication.

12 Deterministic memory

12.1 Memory Partitions

A Memory Partition allocates homogeneous fixed-size blocks from application-supplied storage.

Fixed-size block pool



No variable-size search, coalescing, or heap fragmentation model.

Fixed-block Memory Partition

Allocation and free are immediate; they do not wait or affect task priority. There is no variable-size search, block splitting, coalescing or general heap-fragmentation model.

```
typedef struct
{
    RK_TICK timestamp;
    CHAR text[60];
} LogRecord;

#define LOG_BLOCKS 8U

static RK_MEM_PARTITION logMemory;
RK_DECLARE_MEM_POOL(LogRecord, logPool, LOG_BLOCKS)

VOID InitLogMemory(VOID)
{
    RK_ERR err = kMemPartitionInit(&logMemory,
                                   logPool,
                                   sizeof(LogRecord),
                                   LOG_BLOCKS);
    K_ASSERT(err == RK_ERR_SUCCESS);
}
```

Allocate and test for exhaustion:

```
LogRecord *record = kMemPartitionAlloc(&logMemory);
if (record != NULL)
{
    FillRecord(record);
}
```

Return the block exactly once to its original partition:

```
kMemPartitionFree(&logMemory, record);
```

Block size is rounded up to a word boundary. The pool must be aligned, sized for the rounded block geometry and contain at least one block. Allocation does not clear a reused block; initialise application fields before use.

12.1.1 Mail queue pattern

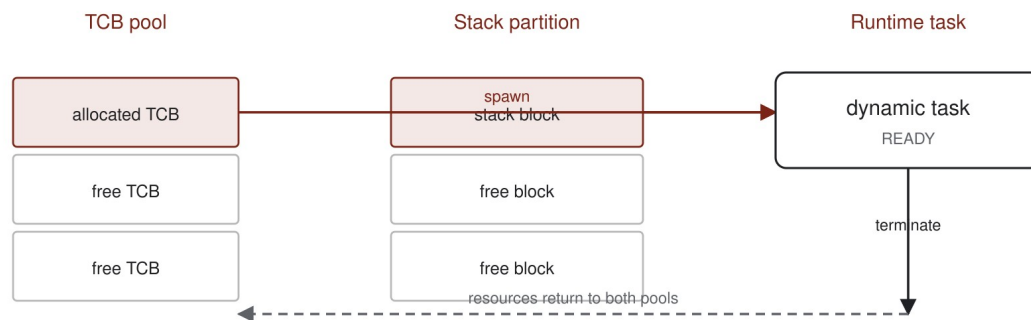
For variable or larger payloads, combine a Memory Partition with a one-word Message Queue of pointers:

1. sender allocates a block;
2. sender fills it and enqueues its pointer;
3. receiver dequeues and processes the block;
4. receiver returns it to the same partition.

This keeps kernel copy cost bounded to one word while making lifetime and ownership explicit. The sender must free the block if a non-blocking queue send fails.

13 Dynamic tasks

Dynamic tasks support middleware that must create and terminate tasks after scheduling has started. They should still come from a bounded, known configuration.



Dynamic task resource lifecycle

Each dynamic task uses:

- one TCB slot from the fixed kernel task pool;
- one stack block from an application-selected Memory Partition.

```

#define DYNAMIC_STACK_WORDS 256U
#define DYNAMIC_TASKS 2U

RK_DECLARE_DYNAMIC_TASK(workerHandle, WorkerTask)
RK_DECLARE_DYNAMIC_STACK_POOL(workerStackMemory,
                               workerStackPool,
                               DYNAMIC_TASKS,
                               DYNAMIC_STACK_WORDS)

static RK_DYNAMIC_TASK_ATTR workerAttributes = {
    .taskFunc = WorkerTask,
    .argsPtr = RK_NO_ARGS,
    .taskName = "DynWork",
    .priority = 3U,
    .preempt = RK_PREEMPT,
    .stackMemPtr = &workerStackMemory
};
  
```

Initialise the stack partition before spawning:

```

kMemPartitionInit(&workerStackMemory,
                  workerStackPool,
  
```

```

        sizeof(workerStackPool[0]),
        DYNAMIC_TASKS);
kTaskSpawn(&workerAttributes, &workerHandle);

```

A successfully spawned task becomes READY immediately and may preempt its creator.

Only runtime-spawned tasks may be terminated. `kTaskTerminate(&handle)` terminates another eligible dynamic task and sets that handle variable to `NULL`. `kTaskTerminateSelf()` defers cleanup to `PostProc` because the running task cannot reclaim its own active stack.

Termination is rejected while the task owns or waits for a resource, remains in an unsupported blocking relation, or has dependants through Port, Channel or Rendezvous state. Copied stale handles remain the application's responsibility even after the handle passed to terminate is cleared.

Real-time rule: runtime creation changes the active task set. Bound when and how many tasks may exist. A dynamic task should use `kSleepUntil()` with a local anchor for periodic work; it did not participate in the scheduler-start phase used by `kSleepRelease()`.

14 Trace and fault handling

14.1 Trace console

When `RK_CONF_TRACE` is enabled, `kTraceInit()` starts an interactive UART-backed trace task. The board supplies a non-blocking receive hook and signals input from its UART ISR.

Name initialised objects so console output is readable:

```

kTraceNameObject(&sampleQueue, "SampleQ");
kTraceNameObject(&stateLock, "StateLk");
kTraceInit();

```

Names are limited by `RK_NAME_SIZE`; with the default size, use at most seven visible characters plus the terminating NUL.

Useful commands include:

Command	Use
top	Task state, effective and nominal priority, dispatches, CPU accounting and stack watermark
list kobjects	Registered kernel objects and last operation
list kmesg	Queue depth, owner and waiting senders or receivers
list kipc	Task-backed Channel and Rendezvous state
list ksema	Semaphore and Mutex state
list kmem	Partition size and free blocks
list ktimers	Application timers
hist <name>	Recent operations for one object
hist task/<name>	Effective-priority changes for one task

Trace CPU percentage is diagnostic accounting, not a cycle-accurate profiler. Use the history with the task and object snapshots to understand causality.

14.2 Error handling

In checked builds, invalid API use can call `kErrorHandler()` in addition to returning an error. Configure the handler for the application's fault policy: halt, capture evidence, reset or hand control to a safety mechanism as appropriate.

Treat these as programming errors rather than routine branch conditions:

- using an uninitialised or wrong object;
- blocking from an ISR;
- recursive Mutex lock;
- unlocking a Mutex without ownership;
- mixing Mutex ownership with Port, Channel or Rendezvous authority;
- freeing a block to the wrong partition;
- letting a task return from its entry function.

Stack overflow and exhausted bounded resources should produce evidence that is useful after reset. The trace stack watermark, object history and application-specific retained fault record are complementary.

15 Choosing a communication service

The services differ mainly in what they retain and where completion occurs.

	Queue	Port	Rendezvous	Channel	MRM
Buffers history	yes	yes	no	no	latest only
Success means	admitted	admitted	copied	reply done	published
Receiver	any	owner	named task	server	many
Priority relation	none*	backpressure	handoff	call duration	none

* A plain owned queue can be configured as a Port; ownership changes the scheduling contract.

Comparison of RK0 message-passing services

Need	Choose	Success means
Count occurrences or units	Semaphore	one token was consumed or posted
Private bitwise occurrences	Task Event Register	requested bits were present and consumed
Wait for a future notification	Sleep Queue	the task was signalled after it began waiting
Preserve a bounded message history	Message Queue	message copied to receiver or queue storage
Keep only one pending newest value	overwrite Mailbox	one-slot value replaced or delivered
Deliver one value to present waiters	broadcast Mailbox	every targeted waiting receiver obtained a copy
Send buffered work to one owner	Port	message admitted to the owner's queue

Need	Choose	Success means
Deliver directly and wait until copied	Rendezvous	receiver copied the fixed-size payload
Invoke a server and receive a result	Channel	server called kChannelDone()
Publish latest state to many readers	MRM	complete value became current

Ask these questions in order:

1. Is this an occurrence, shared-state ownership or a message?
2. If it is a message, should history be preserved, discarded or never created?
3. Does the sender complete on admission, on copy or on server reply?
4. Is there one owner, one receiver, one server or many readers?
5. What priority dependency appears when a more urgent task waits?

Do not emulate a stronger contract with a weaker service unless the application explicitly accepts the changed completion, timeout and scheduling semantics.

16 Compact API index

This index is for orientation. See `kapi.h` for exact parameters, conditional compilation and return values.

Family	Principal operations
Kernel and tasks	<code>kInit</code> , <code>kTaskInit/kCreateTask</code> , <code>kYield</code> , task accessors
Dynamic tasks	<code>kTaskSpawn</code> , <code>kTaskTerminate</code> , <code>kTaskTerminateSelf</code>
Scheduler control	<code>kSchLock</code> , <code>kSchUnlock</code>
Time	<code>kSleepDelay</code> , <code>kSleepRelease</code> , <code>kSleepUntil</code> , <code>kBusyDelay</code> , <code>kTickGet</code>
Callout timer	<code>kTimerInit</code> , <code>kTimerCancel</code>
Task Events	<code>kEventSet</code> , <code>kEventGet</code> , <code>kEventQuery</code> , <code>kEventClear</code>
Semaphore	<code>kSemaphoreInit</code> , <code>kSemaphorePend</code> , <code>kSemaphorePost</code> , <code>kSemaphoreQuery</code>
Sleep Queue	<code>kSleepQueueInit</code> , <code>Wait</code> , <code>Signal</code> , <code>Wake</code> , <code>Ready</code> , <code>BlockReadyTask</code> , <code>Query</code>
Mutex	<code>kMutexInit</code> , <code>kMutexLock</code> , <code>kMutexUnlock</code> , <code>kMutexQuery</code>
Condition helper	<code>kCondVarWait</code> , <code>kCondVarSignal</code> , <code>kCondVarBroadcast</code>
Message Queue	<code>kMesgQueueInit</code> , <code>Send</code> , <code>Recv</code> , <code>Jam</code> , <code>Peek</code> , <code>Reset</code> , <code>Query</code>
Mailbox	<code>kMboxInit</code> , <code>kMboxPost</code> , <code>kMboxPend</code> , broadcast operations, overwrite through <code>kMesgQueuePostOvw</code>
Port	<code>kPortInit</code> , <code>kPortSend</code> , <code>kPortRecv</code> , <code>kPortJam</code> , <code>kPortPostOvw</code> , <code>kPortReset</code>
Rendezvous	<code>kRendezvousInit</code> , <code>kRendezvousSend</code> , <code>kRendezvousRecv</code>
Channel	<code>kChannelCall</code> , <code>kChannelAccept</code> , <code>kChannelDone</code>
MRM	<code>kMRMInit</code> , <code>kMRMReserve</code> , <code>kMRMPublish</code> , <code>kMRMGet</code> , <code>kMRMUnget</code>
Memory Partition	<code>kMemPartitionInit</code> , <code>kMemPartitionAlloc</code> , <code>kMemPartitionFree</code>
Trace	<code>kTraceInit</code> , <code>kTraceNameObject</code> , snapshot and history APIs

17 Further reading

- [RK0 source and public API](#)
- [RK0 Service Map](#)
- [RK0 DocBook](#)
- [Application structure and examples](#)
- [Kernel trace console](#)